Process and Process States

A **process** is a program in execution. A program is a passive file on disk; it becomes a process when it is loaded into memory and given a CPU context. Each process has its own address space made of the **text** (code), **data** (global variables), **heap** (dynamic memory) and **stack** (function calls and local variables).

Process states

- **New**: the process is being created.
- **Ready**: it is waiting in the ready queue to be given the CPU.
- **Running**: its instructions are being executed.
- **Waiting (Blocked)**: it is waiting for an event such as I/O completion.
- **Terminated**: it has finished execution.

Transitions: New → Ready (admitted), Ready → Running (scheduler dispatch), Running → Ready (interrupt or time slice over), Running → Waiting (I/O request), Waiting → Ready (I/O complete), Running → Terminated (exit).

Process Control Block (PCB)

The operating system keeps one PCB per process. It stores the process ID, process state, program counter, CPU registers, scheduling information (priority), memory-management information (page table) and I/O status. During a **context switch** the OS saves the PCB of the running process and loads the PCB of the next one. Context-switch time is pure overhead because no useful work is done.

> **Exam tip:** A process in the Waiting state can never go directly to Running. It must first move to Ready.

Threads and Multithreading

A **thread** is the smallest unit of CPU execution inside a process. Threads of the same process share the code, data, heap and open files, but each thread has its own **program counter, registers and stack**.

Point	Process	Thread
Memory	Separate address space	Shares the process address space
Creation cost	High	Low
Context switch	Slower	Faster
Communication	Needs IPC (pipes, shared memory)	Directly through shared data
Failure	Does not affect other processes	Can crash the whole process

Benefits of multithreading

- **Responsiveness**: a web browser can load a page in one thread while the user scrolls in another.
- **Resource sharing**: threads share memory without extra setup.
- **Economy**: creating and switching threads is cheaper than processes.
- **Scalability**: threads can run in parallel on multicore CPUs.

User-level and kernel-level threads

User-level threads are managed by a library without kernel support, so switching is fast, but one blocking system call blocks all of them. Kernel-level threads are managed by the OS, so one thread can block while others run. Mapping models are many-to-one, one-to-one (used by Linux and Windows) and many-to-many.

CPU Scheduling Algorithms

CPU scheduling decides which ready process gets the CPU next. Criteria: maximise **CPU utilisation** and **throughput**; minimise **turnaround time** (completion − arrival), **waiting time** (turnaround − burst) and **response time**.

- **FCFS** (First Come First Served): non-preemptive; simple but suffers from the **convoy effect** when a long job delays short ones.
- **SJF** (Shortest Job First): gives the minimum average waiting time; the preemptive version is **SRTF**. Problem: burst times must be predicted, and long jobs can starve.
- **Priority**: the highest priority runs first; starvation is solved by **aging** (slowly raising the priority of waiting processes).
- **Round Robin**: each process gets a time quantum, then goes to the back of the queue. Good response time; a very large quantum makes it behave like FCFS.

Worked example (all processes arrive at time 0)

Process	Burst time (ms)
P1	6
P2	8
P3	7
P4	3

- **FCFS** (P1, P2, P3, P4): waiting times 0, 6, 14, 21. Average = 41 / 4 = **10.25 ms**.
- **SJF** (P4, P1, P3, P2): waiting times P4 = 0, P1 = 3, P3 = 9, P2 = 16. Average = 28 / 4 = **7 ms**.
- **Round Robin, quantum 4**: order P1(0–4), P2(4–8), P3(8–12), P4(12–15), P1(15–17), P2(17–21), P3(21–24). Completion: P1 = 17, P2 = 21, P3 = 24, P4 = 15. Waiting = completion − burst: 11, 13, 17, 12. Average = 53 / 4 = **13.25 ms**.

Exam tip: SJF is optimal for average waiting time, which is why it gives the lowest value in the example.

Process Synchronization and Semaphores

When processes share data, the result can depend on the order of execution. This is a **race condition**. The part of the code that accesses shared data is the **critical section**.

Requirements of a critical-section solution

- **Mutual exclusion**: only one process is in the critical section at a time.
- **Progress**: if no process is inside, the choice of who enters next cannot be postponed forever.
- **Bounded waiting**: there is a limit on how many times others can enter before a waiting process gets its turn.

Semaphores

A semaphore S is an integer accessed only through two atomic operations. **wait(S)** (also called P) decrements S and blocks the process if S becomes negative. **signal(S)** (also called V) increments S and wakes a waiting process. A **binary semaphore** (0 or 1) works like a mutex lock. A **counting semaphore** controls access to a resource with several instances, for example 5 printers.

Producer–consumer with a bounded buffer of size n

Use three semaphores: **mutex = 1** for the buffer, **empty = n** for free slots and **full = 0** for filled slots. The producer does wait(empty), wait(mutex), adds an item, signal(mutex), signal(full). The consumer does wait(full), wait(mutex), removes an item, signal(mutex), signal(empty).

> **Exam tip:** Swapping wait(mutex) and wait(empty) in the producer can cause a deadlock when the buffer is full.

Deadlock and the Banker's Algorithm

A **deadlock** is a state in which a set of processes wait forever, each holding a resource that another one needs.

Four necessary conditions (all must hold)

- **Mutual exclusion**
- **Hold and wait**
- **No preemption**
- **Circular wait**

Handling methods: **prevention** (break one condition, for example by ordering resources to stop circular wait), **avoidance** (Banker's algorithm), **detection and recovery** (kill a process or preempt resources), or **ignoring** the problem (the ostrich approach used by most desktop systems).

Banker's algorithm example

Resources A, B, C. Available = (3, 3, 2). Need = Max − Allocation.

Process	Allocation	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Safety check: P1 can finish (1 2 2 ≤ 3 3 2), so Available becomes 5 3 2. Then P3 → 7 4 3, P4 → 7 4 5, P0 → 7 5 5, P2 → 10 5 7. The safe sequence **<P1, P3, P4, P0, P2>** exists, so the system is in a **safe state**.

> **Exam tip:** An unsafe state does not always lead to deadlock, but a safe state never does.

Paging and Address Translation

Paging divides logical memory into fixed-size blocks called **pages** and physical memory into blocks of the same size called **frames**. A process can be placed in any free frames, so memory does not need to be contiguous and there is **no external fragmentation**. A little **internal fragmentation** can occur in the last page.

Address translation

A logical address is split into a **page number (p)** and an **offset (d)**. The page table maps p to a frame number f, and the physical address = f × page size + d.

Example: page size = 1 KB (1024 bytes), logical address = 2500. Page number = 2500 / 1024 = **2**, offset = 2500 mod 1024 = **452**. If page 2 is in frame 5, the physical address = 5 × 1024 + 452 = **5572**.

Translation Lookaside Buffer (TLB)

The page table is kept in memory, so every access would need two memory accesses. The **TLB** is a small, fast cache of recent page-table entries. Effective access time (EAT) = h × (TLB time + memory time) + (1 − h) × (TLB time + 2 × memory time).

Example: hit ratio 80%, TLB 20 ns, memory 100 ns. EAT = 0.8 × 120 + 0.2 × 220 = 96 + 44 = **140 ns**.

> **Exam tip:** Paging removes external fragmentation; segmentation removes internal fragmentation but can cause external fragmentation.

Virtual Memory and Page Replacement

With **virtual memory**, only the pages a process needs are kept in memory (**demand paging**). When a needed page is not in memory, a **page fault** occurs. If no frame is free, a page replacement algorithm chooses a victim page.

- **FIFO**: replaces the oldest page. Simple, but suffers from **Belady's anomaly** (more frames can give more faults).
- **Optimal (OPT)**: replaces the page that will not be used for the longest time. It gives the fewest faults but needs future knowledge, so it is used as a benchmark.
- **LRU** (Least Recently Used): replaces the page not used for the longest time in the past. A good approximation of OPT, with no Belady's anomaly.

Worked example: 3 frames

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2

Algorithm	Page faults
FIFO	10
LRU	9
Optimal	7

Page-fault rate = faults / references. For LRU it is 9 / 13 ≈ 69%.

Thrashing

Thrashing happens when a process does not have enough frames for its working set, so it spends more time paging than executing and CPU utilisation drops sharply. It is controlled with the **working-set model** or **page-fault frequency**, or by reducing the degree of multiprogramming.

Exam tip: Optimal always gives the fewest faults, so it is the lower bound to compare other algorithms against.